

UNIT-2

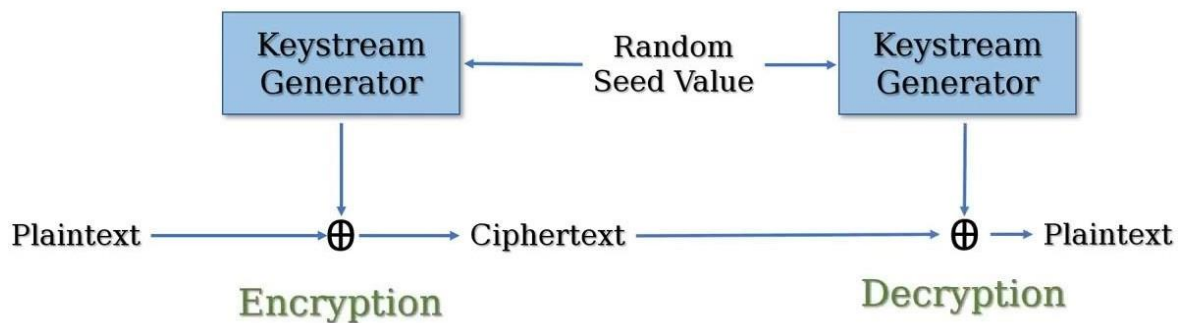
Stream Ciphers and Block Ciphers

In **Information and Network Security**, encryption plays a vital role in protecting data. Two fundamental types of encryption algorithms are **stream ciphers** and **block ciphers**. Both are **symmetric key ciphers**, meaning the same key is used for both encryption and decryption.

1. Stream Ciphers

- **Definition:** Stream ciphers encrypt data one bit or byte at a time.
- **Working:** Operates on a continuous stream of plaintext and encrypts it with a key stream.
- **Features:**
 - Fast and efficient for real-time applications (e.g. voice or video transmission).
 - Suitable for encrypting data of unknown or variable length.
 - Requires **synchronization** between sender and receiver.
- **Examples:** RC4, A5/1 (used in GSM), Salsa20.

Stream Cipher



Advantages:

- Low memory requirements.
- Faster encryption and decryption for continuous data.

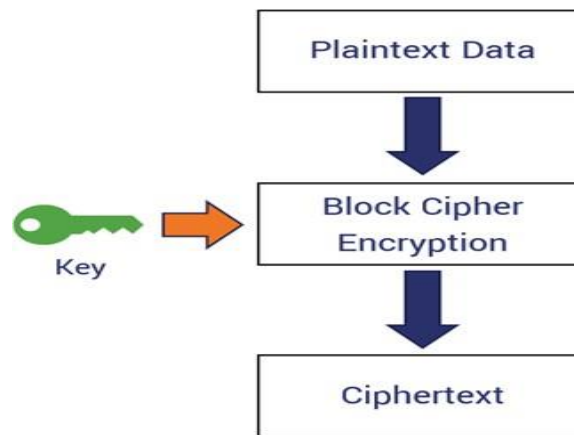
Disadvantages:

- More vulnerable if the key stream is reused.
- If you **use the same secret code(key stream)** to lock **more than one message**, it

becomes **easier for hackers to break** the messages.

2. Block Ciphers

- **Definition:** Block ciphers encrypt data in fixed-size blocks(e.g.,64-bit, 128-bit).
- **Working:** Divides plaintext into blocks and encrypts each block separately using the same key.
- **Features:**
 - **Block ciphers** are usually **better and safer** than **stream ciphers** when it comes to **encrypting files**.
 - Works with **modes of operation**(ECB,CBC,CFB,OFB,CTR)to encrypt data of any length.
- **Examples:** AES, DES, Blowfish, IDEA.



Advantages:

- The system or method is better at **protecting itself from being hacked or misused** by attackers.
- Can handle large data securely.

Disadvantages:

- Slower compared to stream ciphers.
- Padding is required if data doesn't fit exactly into blocks.

Difference between Stream Ciphers and Block Ciphers

Feature	Stream Cipher	Block Cipher
Data Processing	Encrypts one bit or byte at a time	Encrypts a fixed-size block of data (e.g., 64 or 128 bits)
Speed	Faster, suitable for real-time communication	Slower, due to block-by-block processing

Feature	Stream Cipher	Block Cipher
Data Size	Works on data of variable or unknown length	Works best with known fixed-size data
Encryption Method	Uses a key stream to encrypt each bit or byte	Uses the same key to encrypt each data block
Memory Requirement	Requires less memory	Requires more memory
Security	Less secure if key stream is reused	More secure, supports multiple modes of operation
Error Propagation	Errors affect only one bit/byte	Errors may affect entire block
Padding	Not required	Required if data doesn't fill a complete block
Typical Use Cases	Streaming data(e.g. voice, video)	File and message encryption
Examples	RC4, A5/1,Salsa20	AES, DES, Blowfish, IDEA

Block Cipher Structure

In Information and Network Security, a **block cipher** is a symmetric encryption algorithm that operates on fixed-size blocks of plaintext and transforms them into cipher text using a secret key.

□ Basic Structure of Block Cipher

A block cipher typically includes the following components:

1. PlaintextInput

- Data is divided into **fixed-size blocks** (e.g.,64-bit or 128-bit).
- Each block is encrypted independently (as in ECB) or with chaining (as in CBC).

2. Key

- A **single symmetric key** is used for both encryption and decryption.

- The key is **expanded into multiple round keys** through a process known as the **key schedule**.

3. Rounds

- Encryption is performed over multiple rounds(e.g.,10–16 rounds).
- Each round applies a sequence of transformations:
 - **Substitution**(using S-boxes)
 - **Permutation**(rearrangement to f bits)
 - **Mixing and key addition**

4. Substitution and Permutation

- **Substitution:** Replaces bits using S-box to introduce non-linearity.
- **Permutation:** Rearranges bits to spread information (diffusion).

5. Key Mixing

- Round key is XORed with data to create confusion (dependency on key).

6. Cipher text Output

- After the final round, encrypted data (cipher text) is produced.
- Decryption reverses this process using the same key in inverse order

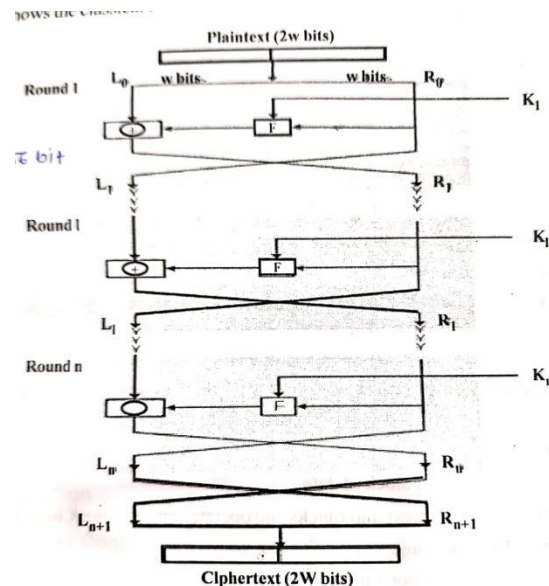


Figure 2.2 Classical Block Structure

□ Modes of Operation

Block ciphers use different modes to encrypt data longer than a single block:

- **ECB(Electronic Code book)**
- **CBC(Cipher Block Chaining Mode)**
- **CFB(Cipher Feedback)**
- **OFB(Output Feedback)**
- **CTR(Counter Mode)**

Mode	Description
ECB	Electronic Code book– Simple but in secure for patterns.
CBC	Cipher Block Chaining–Adds randomness via chaining.
CFB	Cipher Feedback –Converts block cipher into stream cipher.
OFB	Output Feedback – Similar to CFB but more error-resistant.
CTR	Counters Mode–Uses counter and parallelizable; fast and secure.

These modes provide different levels of security and error propagation behavior.

□ Popular Block Ciphers

- **DES(Data Encryption Standard)** –64-bit blocksize,56-bitkey.–now outdated
- **AES(Advanced Encryption Standard)**–128-bitblock,128/192/256-bitkeys.–highly secure
- **Blowfish, IDEA, RC5**–each with unique block/key size and design.

✓ Advantages of Block Cipher

- Provides **strong security** (especially with AES).
- Well-suited for **bulk data encryption**–files, emails, and storage.

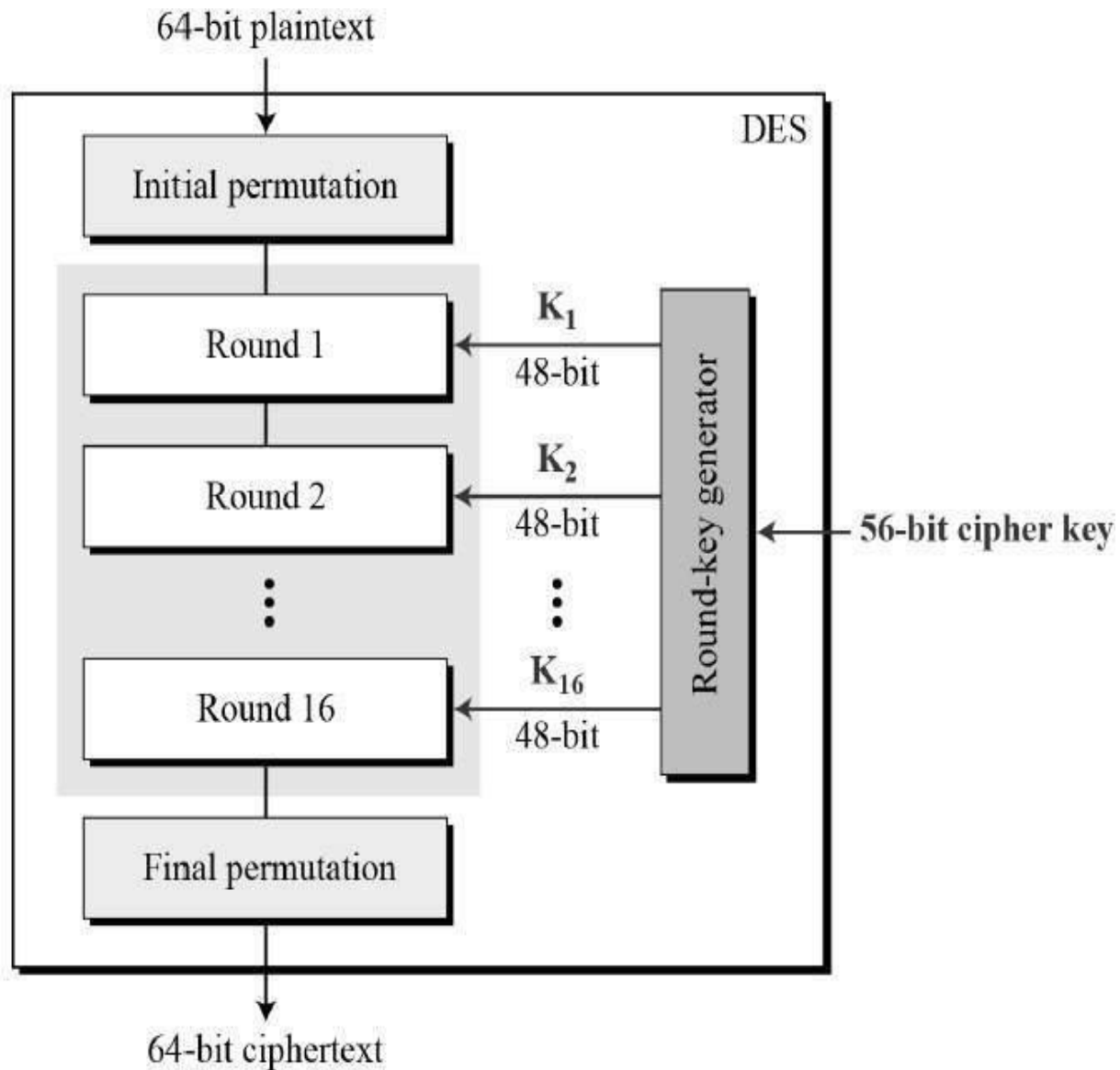
□ Disadvantages

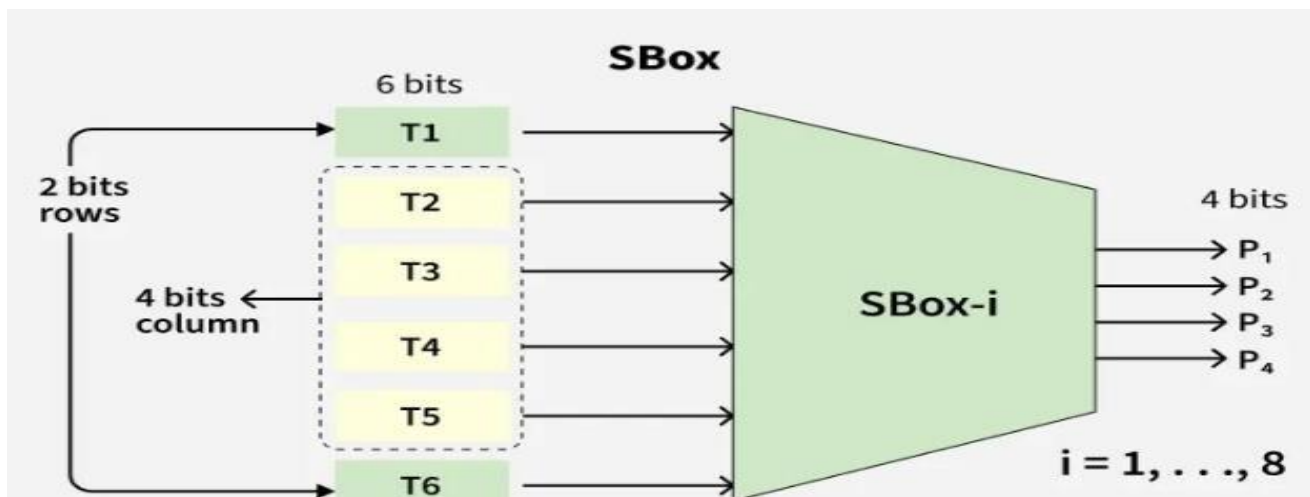
- **Slower** compared to stream ciphers.
- Requires **padding** if plaintext isn't aligned to block size.
- Improper mode usage (e.g., ECB)can expose data patterns.

Data Encryption Standard (DES)

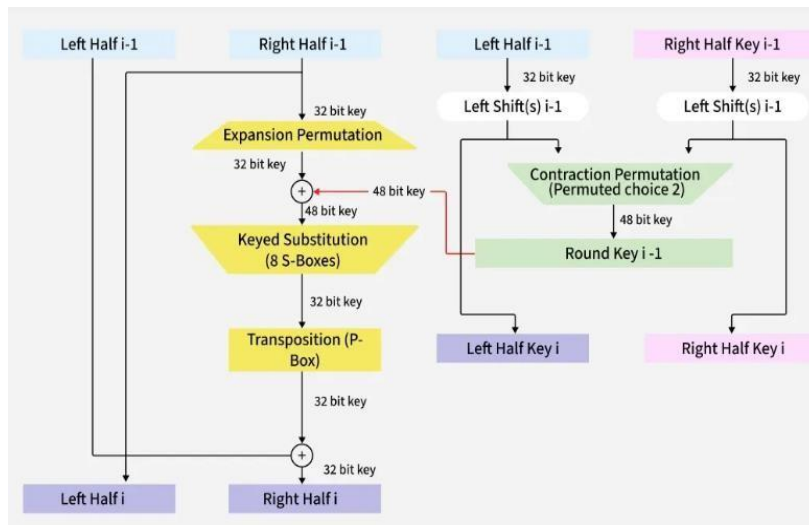
□ Introduction:

DES (Data Encryption Standard) is a **symmetric-key block cipher** developed by IBM in the early 1970s and adopted by the U.S. government as a federal standard in 1977. It became a widely used method for encrypting digital data.





Every round receives 64-bits permuted plaintext from the Initial Permutation function and 48-bit transformed subkey(K_i).The permuted 64-bit plaintext is divided into two halves called as Left Plaintext (LPT) and Right Plaintext (RPT). Both of these halves are 32 bit in size. The right half or Right Plaintext(RPT)is processed using Mangler(F)function. Mangler(F)function involves expansion, key mixing, substitution (S-boxes), and permutation (P-box) of RPT.



□ Features of DES:

Feature	Description
Cipher Type	Symmetric block cipher
Block Size	64bits (plaintext & cipher text)
Key Size	56bits (actual key), 64bits with 8 parity bits
Rounds	16rounds of processing
Mode of Operation	Can use ECB, CBC, CFB, OFB, etc.
Security Level	Considered insecure for modern use due to short key length

□ Working of DES:

1. Initial Permutation (IP):

Rearranges the 64-bit plaintext using a fixed permutation table.

2. 16 Rounds of Encryption:

Each round consists of:

- Splitting data into Left and Right halves.
- Expansion of Right half to 48 bits.
- XOR with round key.
- Substitution using 8 S-boxes (non-linearity).
- Permutation and mixing.
- Swapping halves.

3. Final Permutation (FP):

Inverse of the initial permutation applied to the output of round 16.

4. Cipher text Output:

The result is a 64-bit encrypted block of cipher text.

□ Key Generation in DES:

- A 56-bit key is used to generate **16 sub keys**, each of 48 bits, used in each round.
- Sub keys are derived through permutation and shifting operations.

□ Example:

Let's say:

- **PlaintextBlock:** 01110010 1100101011101000 11011010 10101100 00110110
11010101 01010101 (64 bits)
- **Key:** 00010011 00110100 01010111 01111001 10011011 11011100 11110001 10101011
(64 bits, 56 used)

These binary blocks are passed through:

- Initial Permutation
- 16 rounds using sub keys
- Final Permutation

➔ **Output (Cipher text):** A 64-bit encrypted binary block.

Note: Actual DES encryption requires complex binary computation. In real-world applications, DES encryption is performed by software/hardware tools.

✓ Advantages of DES:

- Was a major milestone in data encryption.
- Fast and efficient in hardware.
- Well-studied and understood.

✗ Limitations of DES:

- **Short key length (56 bits)** makes it vulnerable to **brute-force attacks**.
- Not suitable for securing sensitive data in modern systems.

□ Successor of DES:

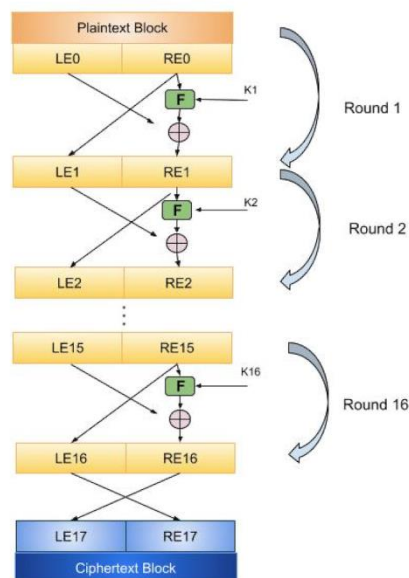
- **Triple DES (3DES):** Applies DES three times to increase security.
- **AES (Advanced Encryption Standard):** Stronger and widely used today.

Feistel Cipher

The Feistel Cipher is not a complete cipher itself, but a design model used to build many block ciphers, such as DES. It provides a simple way to build secure encryption and decryption algorithms.

Key Features of Feistel Cipher

- Works on blocks of data (not on individual characters like substitution ciphers).
- The plaintext block is split into two halves: left (L) and right (R).
- In each round, one half is modified using a function of the other half and a round key.
- Multiple rounds make the cipher stronger.
- Same process is used for both encryption and decryption.



Step-by-Step Algorithm

Here's how a simple 2-round Feistel Cipher works:

1. Convert Plaintext to Binary

- Take the plaintext characters.
- Convert each character to its ASCII value and then into an 8-bit binary string.

2. Divide into Two Halves

- Split the binary string into two equal halves:
- Left half = L1
- Right half = R1

3. Generate Round Keys

- Create random binary keys K1 and K2.
- Each key has a length equal to half the block size.

Round 1 (Encryption)

- Compute function f1:
 $f1 = R1 \text{ XOR } K1$
- Update halves:
 $R2 = L1 \text{ XOR } f1$
 $L2 = R1$

Round 2 (Encryption)

- Compute function f2:

$f2 = R2 \text{ XOR } K2$

- Update halves:
 $R3 = L2 \text{ XOR } f2$
 $L3 = R2$

Final Ciphertext

Concatenate L3 and R3 to get the ciphertext.

Decryption Process

The Feistel Cipher uses the same algorithm for both encryption and decryption, with the round keys simply applied in reverse order.

Example:

Plaintext: Hello

- After encryption → Ciphertext: E1!w(
- After decryption → Retrieved Plaintext: Hello

Plaintext: Geeks

- After encryption → Ciphertext: O;Q
- After decryption → Retrieved Plaintext: Geeks

Why is Feistel Cipher Important?

- It provides a flexible framework to build strong ciphers.
- Many real-world ciphers (like DES) are based on this model.
- Ensures reversibility (you can always get back plaintext from ciphertext).
- Makes encryption/decryption more efficient and practical.

Advanced Encryption Standard(AES)

□ Introduction:

- Advanced Encryption Standard (AES) is a highly trusted **encryption algorithm** used to secure data by converting it into an unreadable format without the proper key.
- It is developed by the National Institute of Standards and Technology (NIST) in 2001.
- It is widely used today as it is much stronger than [DES](#) and triple DES despite being harder to implement.
- **AES encryption** uses various **key lengths** (128, 192, or 256 bits) to provide strong protection against unauthorized access.
- This **data security** measure is efficient and widely implemented in securing **internet communication**, protecting **sensitive data**, and encrypting files.
- AES, a corner stone of modern cryptography, is recognized globally for its ability to keep information safe from cyber threats.

AES is a [Block Cipher](#).

The key size can be 128/192/256 bits. Encrypts data in blocks of 128 bits each.

That means it takes 128 bits as input and outputs 128 bits of encrypted ciphertext. AES relies on the substitution-permutation network principle, which is performed using a series of linked operations that involve replacing and shuffling the input data

Working of the Cipher

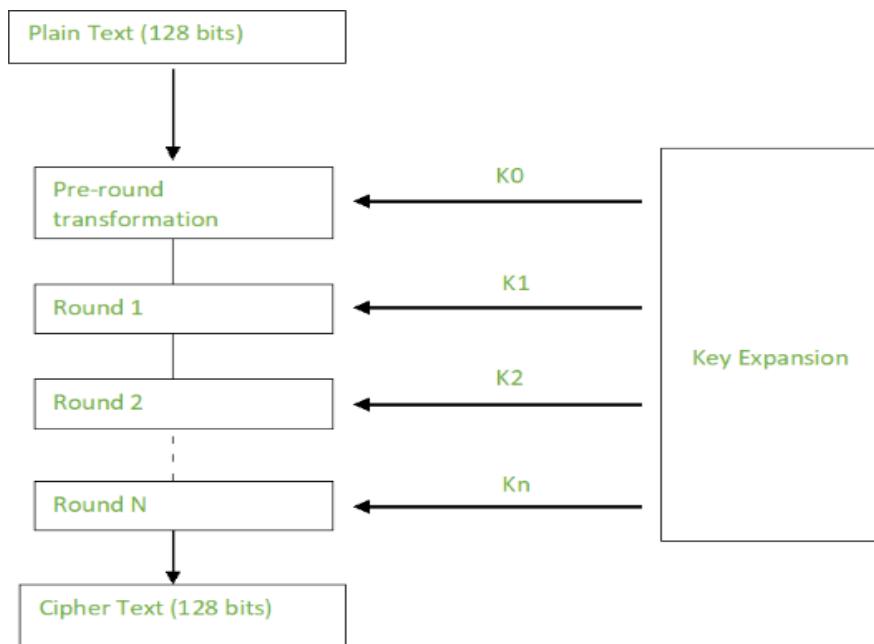
AES performs operations on bytes of data rather than in bits. Since the block size is 128 bits, the cipher processes 128 bits (or 16 bytes) of the input data at a time.

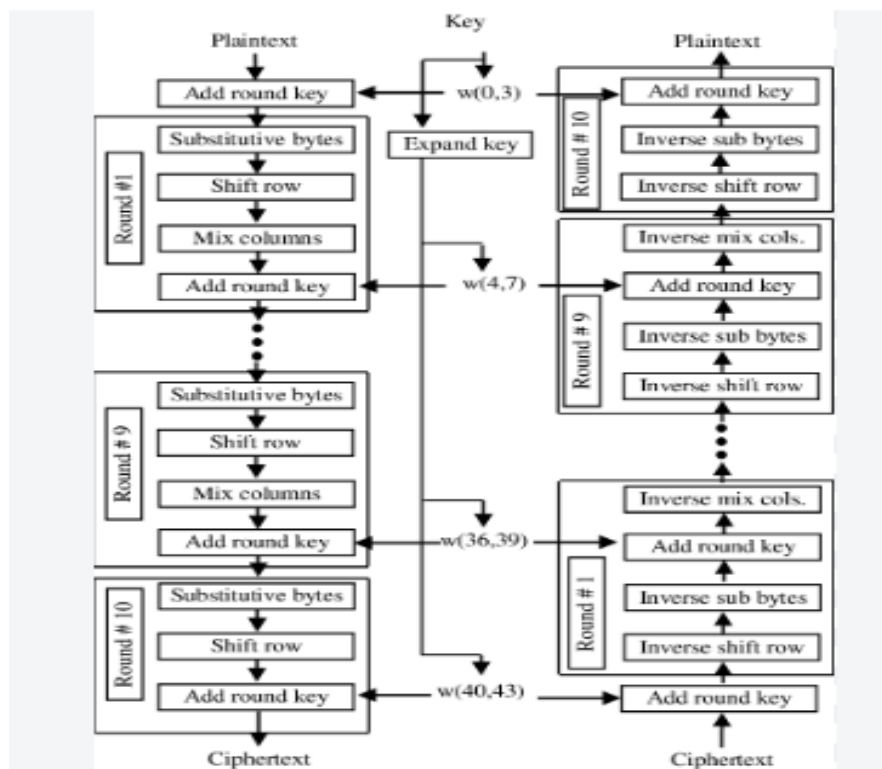
The number of rounds depends on the key length as follows:

N(Number of Rounds)	Key Size (in bits)
10	128
12	192
14	256

Creation of Round Keys

A Key Schedule algorithm calculates all the round keys from the key. So the initial key is used to create many different round keys which will be used in the corresponding round of the encryption.





Each round comprises of 4 steps:

1. Sub Bytes
2. Shift Rows
3. Mix Columns
4. Add Round Key

Step 1. Sub Bytes

This step implements the substitution.

In this step, each byte is substituted by another byte. It is performed using a lookup table also called the S-box. This substitution is done in a way that a byte is never substituted by itself and also not substituted by another byte which is a complement of the current byte. The result of this step is a 16-byte (4 x 4) matrix like before.

The next two steps implement the permutation.

Step 2. Shift Rows

- This step is just as it sounds. Each row is shifted a particular number of times.
- The first row is not shifted
- The second row is shifted once to the left.

- The third row is shifted twice to the left.
- The fourth row is shifted thrice to the left.

Step3: Mix Columns

- This step is a matrix multiplication. Each column is multiplied with a specific matrix and thus the position of each byte in the column is changed as a result.
- This step is skipped in the last round.
- $$\begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Step4: Add Round Keys

- Now the result and output of the previous stage is XOR-ed with the corresponding round key. Here, the 16 bytes are not considered as a grid but just as 128 bits of data.
- After all these rounds 128 bits of encrypted data are given back as output. This process is repeated until all the data to be encrypted undergoes this process.

Applications of AES

AES is widely used in many applications which require secure data storage and transmission. Some common use cases include:

1. **Wireless security:** AES is used in securing wireless networks, such as Wi-Fi networks, to ensure data confidentiality and prevent unauthorized access.
2. **Database Encryption:** AES can be applied to encrypt sensitive data stored in databases. This helps protect personal information, financial records, and other confidential data from unauthorized access in case of a data breach.
3. **Secure communications:** AES is widely used in protocols such as internet communications, email, instant messaging, and voice/video calls. It ensures that the data remains confidential.
4. **Data storage:** AES is used to encrypt sensitive data stored on hard drives, USB drives, and other storage media, protecting it from unauthorized access in case of loss or theft.
5. **Virtual Private Networks (VPNs):** AES is commonly used in VPN protocols to secure the communication between a user's device and a remote server. It ensures that data sent and received through the VPN remains private and cannot be deciphered by eavesdroppers.
6. **Secure Storage of Passwords:** AES encryption is commonly employed to store passwords securely. Instead of storing plaintext passwords, the encrypted version is stored. This adds an extra layer of security and protects user credentials in case of unauthorized access to the storage.
7. **File and Disk Encryption:** AES is used to encrypt files and folders on computers, external storage devices, and cloud storage. It protects sensitive data stored on devices or during data transfer to prevent unauthorized access.

□ Key Features of AES:

Feature	Description
Cipher Type	Symmetric Block Cipher
Block Size	128 bits
Key Sizes	128, 192, or 256 bits
Rounds	10(128-bit key), 12(192-bit), 14(256-bit)
Design	Substitution–Permutation Network (SPN)
Security	Very secure and efficient

▣ Example of AES Parameters

Key Length	Block Size	Number of Rounds
128 bits	128 bits	10
192 bits	128 bits	12
256 bits	128 bits	14

✓ Advantages of AES:

- **Strong Security:** Resistant to all known cryptographic attacks.
- **Fast & Efficient:** Performs well in both hardware and software.
- **Flexible:** Supports different key lengths for varying levels of security.
- **Widely Used:** Used in HTTPS, VPNs, wireless security (WPA2/WPA3), etc.

✗ Limitations of AES:

- Being symmetric, both sender and receiver must securely share the key.
- Implementation flaws (like side-channel attacks) can break AES if not handled securely.

AES Transformation Functions

AES (Advanced Encryption Standard) applies a series of **transformation functions** to convert plaintext into ciphertext through multiple rounds. These transformations are performed on a 4×4 matrix called the **State**.

☒ 1. Sub Bytes (Substitution)

- A **non-linear byte substitution**.
- Each byte in the state matrix is replaced using a fixed **S-box** (Substitution box).
- Provides **confusion** by making the relationship between plaintext and cipher text complex.
- It is an **invertible** operation.

☒ 2. Shift Rows (Permutation)

- This transformation **shifts the rows** of the state matrix to the left:
 - Row 0: no shift
 - Row 1: 1-byte shift
 - Row 2: 2-byte shift
 - Row 3: 3-byte shift
- Provides **diffusion** by rearranging the data within the state.
- Inverse operation: **Inv Shift Rows** (used in decryption).

☒ 3. Mix Columns (Mixing)

- Each column of the state is **transformed using matrix multiplication** over a finite field ($GF(2^8)$).
- It mixes the bytes within each column to further spread the data.
- Provides **inter-column diffusion**.
- Inverse operation: **Inv Mix Columns**.

☒ 4. Add Round Key (Key Addition)

- The **State** matrix is **XORed with the Round Key** generated from the key schedule.
- Ensures that encryption depends on the **secret key**.
- This transformation is performed in **every round**, including the first and last.

□ Round Summary (for 128-bit AES):

Round Type Sub Bytes Shift Rows Mix Columns AddRoundKey

Round Type Sub Bytes Shift Rows Mix Columns Add Round Key

Initial Round	✗	✗	✗	✓
Rounds 1–9	✓	✓	✓	✓
Final Round	✓	✓	✗	✓

Key Expansion in AES :

□ What is Key Expansion?

Key Expansion (also called the **Key Schedule**) is the process used in **AES** to generate a set of **round keys** from the original **cipher key**. These round keys are used in each encryption/decryption round.

□ Key and Round Details

AES Version Cipher Key Size Block Size Number of Rounds Number of Round Keys

AES-128	128 bits (16 bytes)	128 bits	10 rounds	11round keys
AES-192	192 bits (24 bytes)	128 bits	12 rounds	13round keys
AES-256	256 bits (32 bytes)	128 bits	14 rounds	15round keys

Each **round key** is **128bits (16bytes)**, regardless of the key size.

□ Key Expansion Steps (for AES-128)

Step 1: Initial Key

- The original cipher key is divided into **4 words** (each word = 4 bytes = 32 bits).
- Total: 16 bytes = 4 words = W[0] to W[3]

Step 2: Generate 44 Words

- AES-128 requires **44 words** (W[0] to W[43]) to form **11 round keys** (each round key = 4 words = 16 bytes).

Step3:Core Operations Used

To generate $W[i]$, the algorithm uses the following:

1. Rot Word

- Rotates a word by 1 byte to the left.
- Example:
 $\text{Rot Word}([0x09, 0xcf, 0x4f, 0x3c]) \rightarrow [0xcf, 0x4f, 0x3c, 0x09]$

2. Sub Word

- Applies the **S-box substitution** to each byte of a word.
- Example:
 $[0xcf, 0x4f, 0x3c, 0x09] \rightarrow \text{Apply S-box to each byte.}$

3. R con (Round Constant)

- A constant word that varies with each round.
- Example:
 $R\text{ con}[1] = [0x01, 0x00, 0x00, 0x00],$
 $R\text{ con}[2] = [0x02, 0x00, 0x00, 0x00], \text{etc.}$

Word Generation Formula(AES-128):

For $i \geq 4$:

If $i \bmod 4 == 0$:

$W[i] = W[i-4] \text{ XOR } \text{Sub Word}(\text{Rot Word}(W[i-1])) \text{ XOR } R\text{con}[i/4]$ Else:
 $W[i] = W[i-4] \text{ XOR } W[i-1]$

This repeats until all required round keys are generated.

Why Key Expansion is Important

- Ensures **each encryption round uses a different key** derived from the original key.
- Adds security by spreading the influence of the original key through out the rounds.
- Helps protect against key-related attacks.

Example:

Given:

- **Original Cipher Key(128bits)=16bytes=4words**
Let's take a **sample key** (in hexadecimal):

Key=2b7e151628aed2a6abf7876f158809cf This key is

divided into **4 words**:

Word Index	Word(Hex)
W[0]	2b 7e15 16
W[1]	28aed2 a6
W[2]	abf787 6f
W[3]	15 88 09 cf

□ **Step-by-step Key Expansion(First4NewWords)**

$W[4] = W[0] \text{ XOR } \text{Sub Word}(\text{Rot Word}(W[3])) \text{ XOR } \text{Rcon}[1]$

□ Rot Word(W[3])

RotWord(1588 09cf)→8809 cf15

□ SubWord(8809cf15)usingAES-box

Assume Sub Word gives:

Sub Word→ d4 27 11ae

□ Rcon[1]=01 00 00 00

Nowcalculate:

$W[4] = W[0] \text{ XOR } \text{Sub Word}(\text{Rot Word}(W[3])) \text{ XOR } \text{Rcon}[1]$
 $= 2b\ 7e15\ 16 \text{ XOR } d4\ 2711\ ae \text{ XOR } 01\ 00\ 00\ 00$
 $= fa51\ 04\ 98$

$W[5] = W[1] \text{ XOR } W[4]$

$W[5] = 28aed2\ a6 \text{ XOR } fa51\ 04\ 98$
 $= d2\ ffd6\ 3e$

$W[6] = W[2] \text{ XOR } W[5]$

$W[6] = abf7\ 876f \text{ XOR } d2ffd6\ 3e$
 $= 79\ 08\ 51\ 51$

$$W[7] = W[3] \text{ XOR } W[6]$$

$$W[7] = 1588\ 09\ cf \text{ XOR } 7908\ 51\ 51 \\ = 6c80\ 58\ 9e$$

✓ **Resulting Round 1 Key (W[4] to W[7])**

Word Index Word(Hex)

W[4] fa51 04 98

W[5] d2 ffd6 3e

W[6] 79 08 51 51

W[7] 6c80 58 9e

This gives the **round1 key** =

510498 d2 ffd6 3e 79 08 51 51 6c80 589e

The same process continues until **44 words** are generated (for 10 rounds + 1 initial key).

PART-2

Multiple Encryption and Triple DES:

Multiple Encryption

- **Definition:** Multiple encryptions means **encrypting data more than once**, using either the same or different encryption algorithms/keys.
- The main idea is to **increase security** by adding more layers of encryption.

Why it's used:

- To strengthen weak encryption methods(like original DES).
- To make brute-force attacks more difficult.

Triple DES (3DES)

- **Triple DES** is a widely used form of **multiple encryption**, developed to over come the limitations of **Data Encryption Standard (DES)**.
- It **applies the DES algorithm three times** to each data block using either **two** or **three** keys.

What is Triple DES?

Triple DES is an encryption algorithm based on the original Data Encryption Standard(DES).

It is a symmetric encryption algorithm that uses multiple rounds of the Data Encryption Standard (DES) to improve security.

It is also known as Triple DES because it uses the Data Encryption Standard (DES) cipher which takes three times to encrypt its data. It is essentially a block cipher used to encrypt data in 64-bit blocks.

Security-wise, it out performs the original Data Encryption Standard (DES). However, Triple DES is less efficient and slower than the Advanced Encryption Standard (AES).

Features of Triple DES

- It utilizes a triple layer of encryption which means it utilizes three different keys to encrypt the plaintext three times.
- It supports variable key sizes which range from 128bits to 192 bits.
- It basically involves the usage of a symmetric key encryption system, which states that the same key is used for both encryption and decryption.
- It is a block cipher encryption algorithm that works with 64-bit blocks of plaintext at a time.
- It is suitable for legacy systems that require secure encryption.

Encryption Process

The Encryption process of Triple DES involves the following steps:-

Key Generation

This is the first step of the Encryption process of Triple DES. In this step, three unique keys are generated using a key derivation algorithm.

Initial Permutation

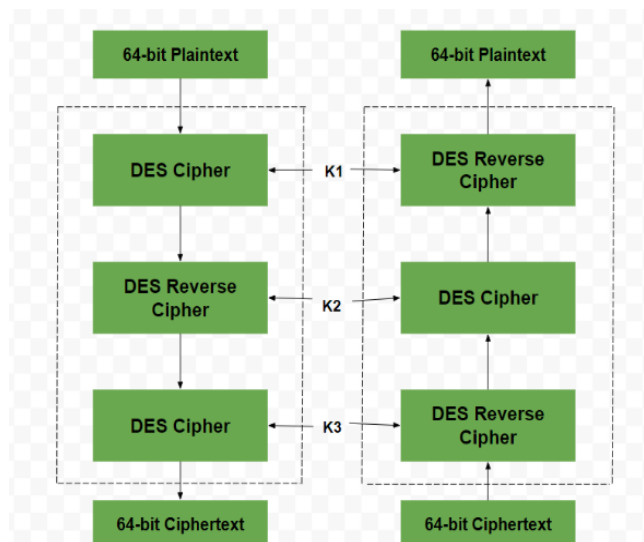
This step comes after the process of Key Generation. It involves the rearrangement of the bits of the plaintext according to a predefined permutation table.

Three Rounds of Encryption

This is regarded as the most important round of the encryption process of Triple DES. It consists of multiple rounds typically 48 rounds in total. In this step, the plaintext is processed three times and gets encrypted, each time we take use of a different key, to create three layers of encryption.

Final Permutation

It completes the Triple [DES encryption](#) process. In this step, the resulting cipher text block under goes a final permutation(FP)operation, which is the inverse of the initial permutation. It returns the bits of the cipher text block to their original order.



Advantages of Triple DES

- It provides three layered encryption technique which provides enhanced security features.
- It offers backward compatibility with Data Encryption Standard which means it can use legacy system that DES uses.
- It supports variable key sizes, which led to enhanced security.
- It is widely used encryption algorithm and is used with many [encryption](#) standards and protocols.

Applications of Triple DES

Financial Transactions: Triple DES is widely used in financial transactions because it secures the transaction that takes place like online banking, credit card payment, etc.

Data Protection: Triple DES is often used to protect sensitive data which are stored on computers, servers, and other electronic devices. It is used in various fields such as healthcare, government sector, etc.

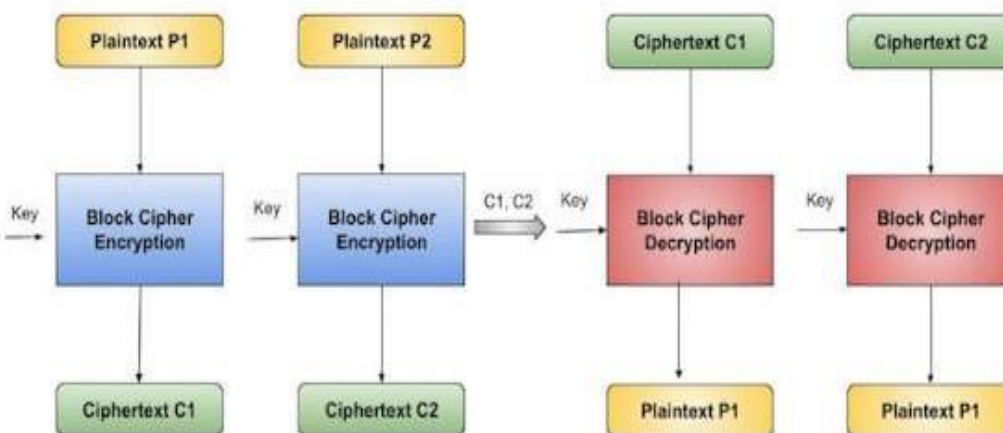
Virtual Private Networks: Triple DES is used to secure the communication process between remote locations. It is done by securing the virtual private networks.

Authentication and Digital Signatures: Triple DES can be used in combination with cryptographic hash functions for generating [digital signatures](#) and verifying the authenticity of digital documents and messages.

Electronic Code Book (ECB)

Definition:

Electronic Code Book (ECB) is the simplest mode of operation for a block cipher, such as DES or AES. In ECB, the plaintext is divided into fixed-size blocks, and each block is encrypted separately using the same secret key.



How ECB Works:

1. The plaintext is split into equal-sized blocks(e.g.,64or128 bits).
2. Each block is encrypted independently using the block cipher algorithm and the same key.
3. The cipher text is formed by joining the encrypted blocks in sequence.

Advantages:

- Simple to understand and implement.
- Allows parallel encryption and decryption of blocks.

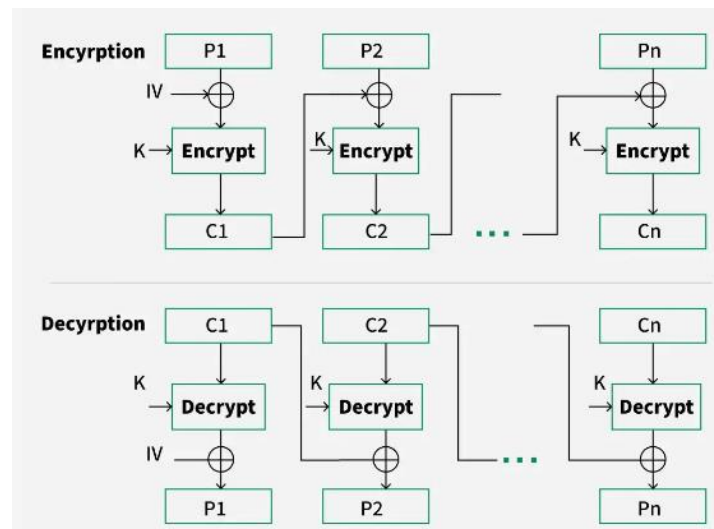
Disadvantages:

- Identical plaintext blocks produce identical cipher text blocks, making patterns visible.
- Not suitable for encrypting large amounts of data where repetition may occur.

Cipher Block Chaining(CBC) Mode

Definition:

Cipher Block Chaining (CBC) is a block cipher mode of operation that introduces chaining between blocks to improve security. In CBC, each plaintext block is XORed with the previous ciphertext block before encryption. This makes each ciphertext block dependent on all previous plaintext blocks.



How CBC Works:

1. An **Initialization Vector(IV)**, which is a random block, is used for the first block to ensure uniqueness.
2. The first plaintext block is XORed with the IV, then encrypted with the key.
3. For each subsequent block, the plaintext block is XORed with the previous ciphertext block before encryption.
4. During decryption, each decrypted block is XORed with the previous ciphertext block (or IV for the first block) to retrieve the plaintext.

Advantages:

- Identical plaintext blocks produce different cipher text blocks , hiding patterns.
- Errors do not spread to all blocks (only the current and next block are affected).

Disadvantages:

- Encryption can not be parallelized due to chaining dependency.
- Requires a unique, unpredictable IV for security

Encryption:

$$P1 \oplus IV \rightarrow E(K) \rightarrow C1$$

$$P2 \oplus C1 \rightarrow E(K) \rightarrow C2$$

$$P3 \oplus C2 \rightarrow E(K) \rightarrow C3$$

...

Decryption:

$$D(K)(C1) \oplus IV \rightarrow P1$$

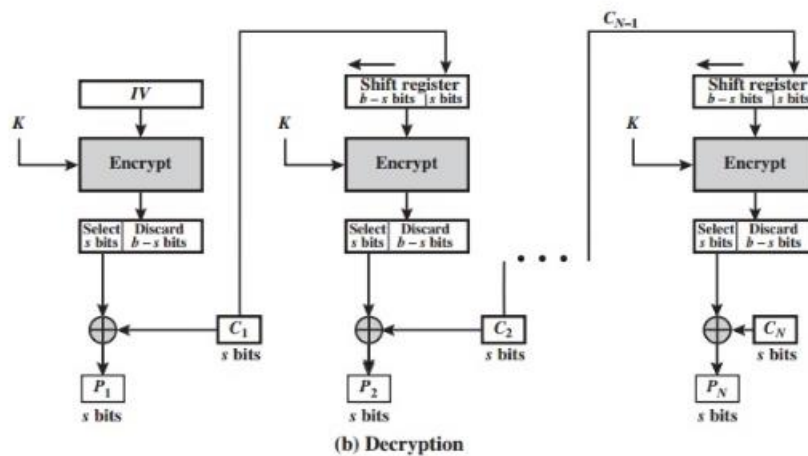
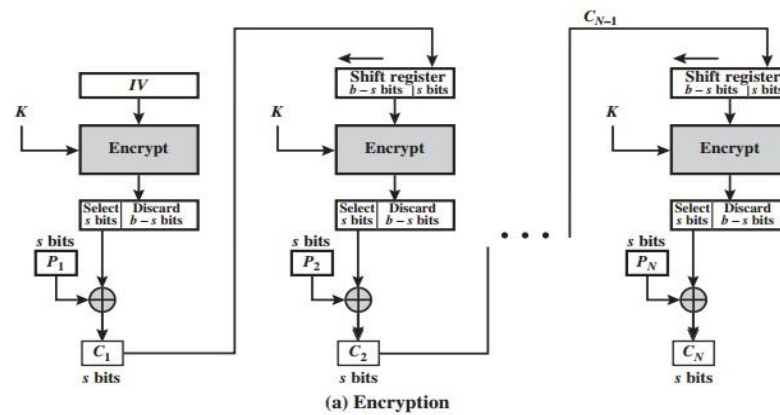
$$D(K)(C2) \oplus C1 \rightarrow P2$$

$$D(K)(C3) \oplus C2 \rightarrow P3$$

Cipher Feedback (CFB) Mode

Definition:

Cipher Feedback (CFB) is a block cipher mode of operation that turns a block cipher into a self-synchronizing stream cipher. Instead of encrypting the plaintext directly, CFB encrypts the previous cipher text (or Initialization Vector for the first block) and then XORs the result with the plaintext to produce the cipher text.



How CFB Works:

1. An **Initialization Vector (IV)** is used for the first step.
2. The IV is encrypted using the block cipher and the secret key.
3. The output of the encryption is XORed with the plaintext block(or segment) to produce the cipher text.
4. The cipher text block becomes the input for encrypting the next plaintext block.
5. Decryption follows the same process because XOR is reversible.

Advantages:

- Can work in smaller units than the block size(e.g., 8-bit segments).
- Does not require padding when plaintext is not a multiple of block size.
- Self-synchronizing: if a bit is lost or corrupted; only a small portion of the message is affected.

Disadvantages:

- Encryption can not be fully parallelized due to chaining.
- Transmission errors propagate for a few blocks

Encryption:

$IV \rightarrow E(K) \rightarrow \text{Output} \rightarrow \text{XOR with } P_1 \rightarrow C_1$

$C_1 \rightarrow E(K) \rightarrow \text{Output} \rightarrow \text{XOR with } P_2 \rightarrow C_2$

$\rightarrow E(K) \rightarrow \text{Output} \rightarrow \text{XOR with } P_3 \rightarrow C_3$

...

Decryption (same as encryption process):

$IV \rightarrow E(K) \rightarrow \text{Output} \rightarrow \text{XOR with } C_1 \rightarrow P_1$

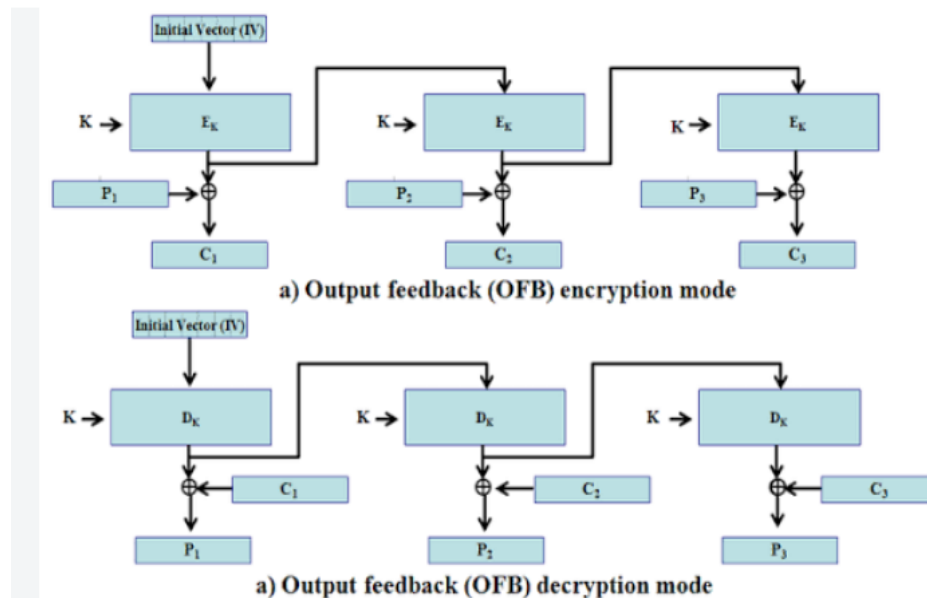
$C_1 \rightarrow E(K) \rightarrow \text{Output} \rightarrow \text{XOR with } C_2 \rightarrow P_2$

$C_2 \rightarrow E(K) \rightarrow \text{Output} \rightarrow \text{XOR with } C_3 \rightarrow P_3$

Output Feedback (OFB) Mode

Definition:

Output Feedback (OFB) is a block cipher mode of operation that converts a block cipher into a synchronous stream cipher. Instead of encrypting plaintext directly, OFB repeatedly encrypts an initial value (Initialization Vector, IV) to produce a key stream, which is then XORed with the plaintext to get cipher text.



How OFB Works:

1. An **Initialization Vector(IV)** is encrypted using the block cipher and key.
2. The encryption output becomes the next input to the block cipher.
3. Each encryption output is XORed with the plaintext to produce the cipher text.
4. The same key stream is used in decryption by XOR in get with the ciphertext to recover the plaintext.

Advantages:

- Errors do not propagate; a single-bit error only affects that bit in the plaintext.
- Can be pre-computed since the key stream is independent of plaintext.
- No padding required for plaintext lengths that are not multiples of blocksize.

Disadvantages:

- Requires a unique IV for each encryption; reusing the same IV with the same key is insecure.
- Not self-synchronizing—loss of synchronization in the stream causes permanent corruption.

Use Cases:

- Secure communications over error-prone channels.
- Applications needing lower or propagation, like satellite and wireless communication.

Encryption:

$IV \rightarrow E(K) \rightarrow O1 \rightarrow \text{XOR with } P1 \rightarrow C1$

$O1 \rightarrow E(K) \rightarrow O2 \rightarrow \text{XOR with } P2 \rightarrow C2 \quad O2 \rightarrow$

$E(K) \rightarrow O3 \rightarrow \text{XOR with } P3 \rightarrow C3$

...

Decryption:

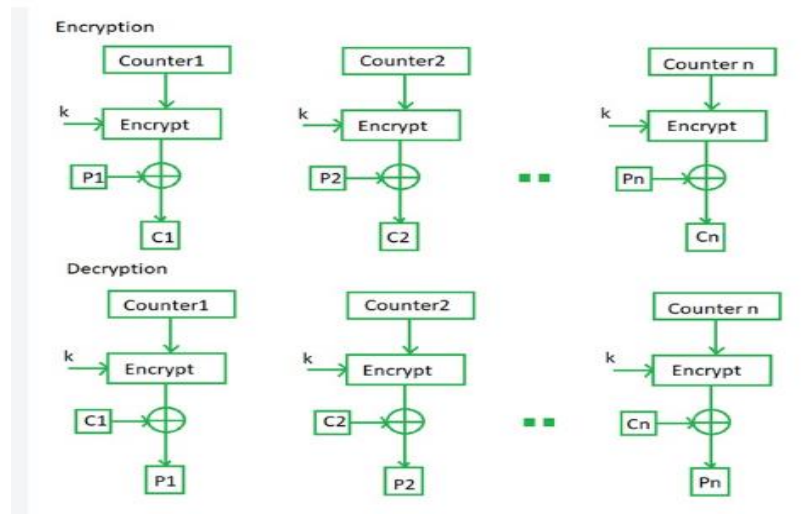
Same process as encryption:

$IV \rightarrow E(K) \rightarrow O1 \rightarrow \text{XOR with } C1 \rightarrow P1 \quad O1 \rightarrow E(K) \rightarrow O2 \rightarrow \text{XOR with } C2 \rightarrow P2$

Counter (CTR) Mode

Definition:

Counter (CTR) mode is a block cipher mode of operation that turns a block cipher into a stream cipher by encrypting sequential counter values. The output of these encrypted counters forms a key stream, which is XORed with the plaintext to produce cipher text.



How CTR Works:

1. A **nonce** (unique value for each encryption) and an initial counter value are combined to form the first input block.
2. The block cipher encrypts this value using the secret key to produce a key stream block.
3. The key stream block is XORed with the plaintext block to get the cipher text block.
4. The counter is incremented for each block and encrypted again to produce the next key stream block.
5. Decryption follows the same process, since XOR is reversible.

Advantages:

- Supports parallel processing for both encryption and decryption.
- No padding required.
- Errors do not propagate; only the affected bits are corrupted.
- Pre-computation of key stream is possible for efficiency.

Disadvantages:

- Re using the same key and once combination is insecure.
- Requires careful management of counter values to prevent repetition.

Encryption:

Nonce||Counter1 $\rightarrow$ E(K) $\rightarrow$ O1 $\rightarrow$ XORwithP1 $\rightarrow$ C1

Nonce||Counter2 $\rightarrow$ E(K) $\rightarrow$ O2 $\rightarrow$ XORwithP2 $\rightarrow$ C2

Nonce||Counter3 $\rightarrow$ E(K) $\rightarrow$ O3 $\rightarrow$ XORwithP3 $\rightarrow$ C3

...

Decryption (same as encryption):

Nonce||Counter1 $\rightarrow$ E(K) $\rightarrow$ O1 $\rightarrow$ XORwithC1 $\rightarrow$ P1

Nonce||Counter2 $\rightarrow$ E(K) $\rightarrow$ O2 $\rightarrow$ XORwithC2 $\rightarrow$ P2